Longest Common Subsequence (LCS) — Module 6 Lab

Course: CSC 821 — Design and Analysis of Algorithms
Group: Group 1
Task: Implement a dynamic-programming solution that finds the longest common subsequence of two strings.

Objective

A **subsequence** is what is left after deleting zero or more characters from a string **without reordering** the ones that remain. The **longest common subsequence** of `s1` and `s2` is the longest string that is a subsequence of both.

The deliverable is a function `lcs(s1, s2)` that returns the **length** of that subsequence, built with a **dynamic-programming** table rather than by searching every possibility.

1. Subsequence is not substring

This distinction is the whole reason the problem is interesting:

	must be contiguous?	`"AGGTAB"` vs `"GXTXAYB"`
substring	yes	longest common substring is `"A"` — length 1
subsequence	no, only the *order* must hold	longest common subsequence is `"GTAB"` — length 4

`G`, `T`, `A`, `B` appear in that order in both strings, just with other characters scattered between them. Because gaps are allowed, brute force is hopeless: a string of length `m` has `2**m` subsequences, so checking them all against `s2` is exponential.

2. The recurrence

Dynamic programming applies here because the problem has the two properties that make a table pay off.

Optimal substructure — the answer for two strings is built from the answers for their own prefixes. Compare the last characters of the prefixes `s1[:i]` and `s2[:j]`:

- **They match.** That character can safely be placed at the end of the LCS, and the rest of the answer is the LCS of the two shorter prefixes: `dp[i][j] = dp[i-1][j-1] + 1`
- **They differ.** They cannot both end the LCS, so at least one of them goes unused. Try dropping each in turn and keep the better result: `dp[i][j] = max(dp[i-1][j], dp[i][j-1])`

Overlapping subproblems — plain recursion on that rule asks for the same prefix pair again and again along different branches. There are only `(m+1) × (n+1)` distinct prefix pairs, so rather than recomputing them we fill each one **once** into a table and read neighbours back in `O(1)`.

Base case. Row `0` and column `0` are all zeros: an empty prefix has nothing in common with anything. Every other cell depends only on the cell above, the cell to the left, and the cell diagonally up-left — all of them already filled if we sweep row by row, left to right.

The answer sits in the bottom-right cell, `dp[len(s1)][len(s2)]`.

In [1]:
```python
def lcs_table(s1, s2):
    """Fill the (m+1) x (n+1) DP table; dp[i][j] = LCS length of s1[:i] and s2[:j].
    m, n = len(s1), len(s2)

    # Row 0 and column 0 stay 0 -- the base case: an empty prefix shares nothing.
    dp = [[0] * (n + 1) for _ in range(m + 1)]

    for i in range(1, m + 1):
        for j in range(1, n + 1):
            if s1[i - 1] == s2[j - 1]:
                dp[i][j] = dp[i - 1][j - 1] + 1             # match -> extend the d
            else:
                dp[i][j] = max(dp[i - 1][j], dp[i][j - 1])  # no match -> drop one

    return dp
```

`lcs` is then just the bottom-right cell — the required function, one line on top of the table:

In [2]:
```python
def lcs(s1, s2):
    """Length of the longest common subsequence of s1 and s2."""
    return lcs_table(s1, s2)[len(s1)][len(s2)]


lcs("AGGTAB", "GXTXAYB")
```

Out[2]:
```
4
```

3. Watching the table fill

Printing the table makes the recurrence visible. Each cell is either *(diagonal + 1)* where the row and column characters agree, or the larger of *(above, left)* where they do not.

In [3]:
```python
def show_table(s1, s2):
    """Print the DP table, using '-' for the empty prefix."""
    dp = lcs_table(s1, s2)
    print("     " + "".join(f"{ch:>4}" for ch in "-" + s2))
    for i, row in enumerate(dp):
        label = "-" if i == 0 else s1[i - 1]
        print(f"{label:>5}" + "".join(f"{value:>4}" for value in row))


show_table("AGGTAB", "GXTXAYB")
```

```
      -   G   X   T   X   A   Y   B
  -   0   0   0   0   0   0   0   0
  A   0   0   0   0   0   1   1   1
  G   0   1   1   1   1   1   1   1
  G   0   1   1   1   1   1   1   1
  T   0   1   1   2   2   2   2   2
  A   0   1   1   2   2   3   3   3
  B   0   1   1   2   2   3   3   4
```

Read along the last row: the value only ever **stays the same or rises by one** as we move right, and it rises exactly where a character is matched. The bottom-right `4` is the length of `"GTAB"`.

Notice too that no value ever decreases going down or right — extending a prefix can only give the LCS more to work with, never less. That monotonicity is what lets the traceback in the next section follow the numbers back greedily.

4. Recovering the subsequence itself

The table holds only lengths, but the characters can be read back out by walking **backwards** from the bottom-right corner:

- If the two characters match, that character belongs in the LCS — record it and step diagonally.
- Otherwise move to whichever neighbour, above or left, the value came from.

The characters come out in reverse order, so the walk finishes with a flip. Where several distinct subsequences share the maximum length, the tie-break below returns one of them.

In [4]:
```python
def lcs_string(s1, s2):
    """Recover one longest common subsequence by walking the table backwards."""
    dp = lcs_table(s1, s2)
    i, j = len(s1), len(s2)
    out = []

    while i > 0 and j > 0:
        if s1[i - 1] == s2[j - 1]:            # this character is part of the LCS
            out.append(s1[i - 1])
            i, j = i - 1, j - 1
        elif dp[i - 1][j] >= dp[i][j - 1]:    # follow the value back to where it c
```

```
            i -= 1
        else:
            j -= 1

    return "".join(reversed(out))             # we walked backwards, so flip it


for a, b in [("AGGTAB", "GXTXAYB"), ("ABCBDAB", "BDCABA")]:
    print(f"{a} vs {b}  ->  length {lcs(a, b)}, subsequence {lcs_string(a, b)!r}")
```

```
AGGTAB vs GXTXAYB  ->  length 4, subsequence 'GTAB'
ABCBDAB vs BDCABA  ->  length 4, subsequence 'BCBA'
```

`"ABCBDAB"` and `"BDCABA"` have **more than one** LCS of length 4 — `"BCBA"`, `"BDAB"` and `"BCAB"` all qualify. The length is unique; the subsequence achieving it need not be.

5. Why the table is worth building

The same recurrence written as plain recursion is correct but recomputes the identical prefix pair over and over. Counting the calls shows the blow-up against the `(m+1) × (n+1)` cells the DP fills.

In [5]:
```
calls = 0

def lcs_naive(s1, s2):
    """The same rule with no table -- exponential, because subproblems repeat."""
    global calls
    calls += 1
    if not s1 or not s2:
        return 0
    if s1[-1] == s2[-1]:
        return lcs_naive(s1[:-1], s2[:-1]) + 1
    return max(lcs_naive(s1[:-1], s2), lcs_naive(s1, s2[:-1]))


a, b = "AGGTABXY", "GXTXAYBZ"
calls = 0
length = lcs_naive(a, b)

print(f"naive recursion : {calls} calls        -> {length}")
print(f"DP table        : {(len(a) + 1) * (len(b) + 1)} cells filled -> {lcs(a, b)}")
```

```
naive recursion : 4614 calls        -> 4
DP table        : 81 cells filled -> 4
```

Same answer, and the gap widens fast — the recursion roughly doubles its work for every character added, while the table grows only as `m × n`.

6. Two applications

Sequence alignment. In bioinformatics the LCS of two DNA strands measures how much of one is preserved in the other despite insertions and deletions.

Diff. `git diff` is LCS underneath: the lines common to both files, in order, are the ones left unmarked — everything outside the LCS is reported as a deletion or an addition.

In [6]:
```
strand_1 = "ACCGGTCGAGTGCGCGGAAGCCGGCCGAA"
strand_2 = "GTCGTTCGGAATGCCGTTGCTCTGTAAA"

common = lcs_string(strand_1, strand_2)
print(f"strand 1  : {strand_1}  ({len(strand_1)} bases)")
print(f"strand 2  : {strand_2}  ({len(strand_2)} bases)")
print(f"LCS       : {common}  (length {lcs(strand_1, strand_2)})")
print(f"similarity: {len(common) / max(len(strand_1), len(strand_2)):.1%} of the lo
```

```
strand 1  : ACCGGTCGAGTGCGCGGAAGCCGGCCGAA  (29 bases)
strand 2  : GTCGTTCGGAATGCCGTTGCTCTGTAAA  (28 bases)
LCS       : GTCGTCGGAAGCCGGCCGAA  (length 20)
similarity: 69.0% of the longer strand
```

In [7]:
```
old = ["import os", "def load(path):", "    data = read(path)", "    return data"]
new = ["import os", "import sys", "def load(path):", "    return read(path)"]

# The same table on lists of lines instead of characters -- the algorithm does not
dp = lcs_table(old, new)
i, j = len(old), len(new)
diff = []
while i > 0 or j > 0:
    if i > 0 and j > 0 and old[i - 1] == new[j - 1]:
        diff.append("  " + old[i - 1])        # kept: part of the LCS
        i, j = i - 1, j - 1
    elif j > 0 and (i == 0 or dp[i][j - 1] >= dp[i - 1][j]):
        diff.append("+ " + new[j - 1])        # only in the new file
        j -= 1
    else:
        diff.append("- " + old[i - 1])        # only in the old file
        i -= 1

print("\n".join(reversed(diff)))
```

```
  import os
+ import sys
  def load(path):
-     data = read(path)
-     return data
+     return read(path)
```

The two unchanged lines are exactly the LCS of the two files; everything else is marked `+` or `-`. Note that `lcs_table` was handed **lists of strings** rather than strings and needed no change — it only ever compares elements with `==`.

7. Cutting the space down to one row

Filling row `i` only ever reads row `i-1` and the cell immediately to the left. Older rows are never touched again, so the whole table need not be kept: two rows are enough, which drops the space from `O(m·n)` to `O(min(m, n))`.

The trade-off is that the traceback of section 4 becomes impossible — with the table discarded there is no path left to walk back along. Use this version when only the **length** is needed.

In [8]:
```
def lcs_two_rows(s1, s2):
    """Same length in O(min(m, n)) space -- but no traceback, the table is thrown a
    if len(s2) > len(s1):
        s1, s2 = s2, s1                        # keep the shorter string as the row

    previous = [0] * (len(s2) + 1)
    for ch1 in s1:
        current = [0] * (len(s2) + 1)
        for j, ch2 in enumerate(s2, start=1):
            if ch1 == ch2:
                current[j] = previous[j - 1] + 1
            else:
                current[j] = max(previous[j], current[j - 1])
        previous = current

    return previous[-1]


for a, b in [("AGGTAB", "GXTXAYB"), ("ABCBDAB", "BDCABA"), (strand_1, strand_2)]:
    print(f"full table {lcs(a, b):>3}   two rows {lcs_two_rows(a, b):>3}   agree: {
```

```
full table   4   two rows   4   agree: True
full table   4   two rows   4   agree: True
full table  20   two rows  20   agree: True
```

8. Edge cases

The base row and column handle the degenerate inputs on their own — no special-casing is needed anywhere in the function.

In [9]:
```
tests = [
    ("AGGTAB", "GXTXAYB", 4),   # the classic pair            -> "GTAB"
    ("ABCBDAB", "BDCABA", 4),   # several LCSs of equal length -> "BCBA"
    ("", "ANYTHING", 0),        # empty string: nothing in common
    ("", "", 0),                # both empty
    ("SAME", "SAME", 4),        # identical: the LCS is the whole string
    ("ABC", "XYZ", 0),          # disjoint alphabets
    ("ABC", "CBA", 1),          # same letters, reversed -> order matters
    ("A", "A", 1),              # single character
]

for s1, s2, expected in tests:
    got = lcs(s1, s2)
    status = "ok" if got == expected else "FAIL"
```

```
    print(f"lcs({s1!r:>10}, {s2!r:>10}) = {got}   expected {expected}   {status:<5}

print()
print("all passed:", all(lcs(s1, s2) == expected for s1, s2, expected in tests))
```

```
lcs(  'AGGTAB',  'GXTXAYB') = 4   expected 4   ok    -> 'GTAB'
lcs( 'ABCBDAB',   'BDCABA') = 4   expected 4   ok    -> 'BCBA'
lcs(        '', 'ANYTHING') = 0   expected 0   ok    -> ''
lcs(        '',         '') = 0   expected 0   ok    -> ''
lcs(    'SAME',     'SAME') = 4   expected 4   ok    -> 'SAME'
lcs(     'ABC',      'XYZ') = 0   expected 0   ok    -> ''
lcs(     'ABC',      'CBA') = 1   expected 1   ok    -> 'A'
lcs(       'A',        'A') = 1   expected 1   ok    -> 'A'

all passed: True
```

`("ABC", "CBA")` is the one worth pausing on: the two strings use the *same three letters*, yet the LCS is only length 1. A subsequence may skip characters but may never reorder them.

9. Confirming the `O(m·n)` cost

If the running time really is proportional to the number of cells, then the time divided by the cell count should stay **flat** as the strings grow — and doubling both lengths, which quadruples the cell count, should cost about **four times** as much.

Wall-clock timings are noisy, so each size below is the **best of three runs**: the minimum is the sample least disturbed by other work on the machine. The `ns/cell` column is the one to watch, as it is far steadier than the doubling ratio.

In [10]:
```
import random
import time

random.seed(821)                                    # fixed seed so the strings are r


def time_lcs(size, repeats=3):
    """Best of `repeats` runs -- the minimum is the sample least disturbed by other
    s1 = "".join(random.choice("ACGT") for _ in range(size))
    s2 = "".join(random.choice("ACGT") for _ in range(size))

    best = float("inf")
    for _ in range(repeats):
        start = time.perf_counter()
        lcs(s1, s2)
        best = min(best, time.perf_counter() - start)
    return best


print(f"{'length':>7}{'cells':>12}{'seconds':>10}{'ratio':>8}{'ns/cell':>10}")
previous_time = None
for size in [200, 400, 800, 1600]:
    elapsed = time_lcs(size)
```

```
    cells = (size + 1) ** 2

    ratio = "-" if previous_time is None else f"{elapsed / previous_time:.1f}x"
    print(f"{size:>7}{cells:>12}{elapsed:>10.3f}{ratio:>8}{elapsed / cells * 1e9:>1
    previous_time = elapsed
```

```
length       cells   seconds   ratio   ns/cell
   200       40401     0.010       -       238
   400      160801     0.053    5.5x       328
   800      641601     0.255    4.8x       397
  1600     2563201     1.011    4.0x       394
```

Across a **64-fold** increase in table size the cost per cell moves only from roughly 240 ns to roughly 395 ns — it drifts up as the table outgrows the CPU cache, then flattens. That is the claim `O(m·n)` makes: one constant-time cell fill per `(i, j)` pair, with nothing about the algorithm itself degrading as the input grows. The doubling ratios land near the predicted 4x but wander either side of it, because cache behaviour and interpreter overhead move the *constant factor* — precisely the thing big-O notation sets aside. Contrast section 5, where the exponential version's work grew by a factor of thousands over a handful of extra characters.

Conclusion

- **Correctness:** every test in section 8 matches its expected length, and the recovered subsequences are genuine subsequences of both inputs.
- **Why DP works here:** the problem has **optimal substructure** (the answer for a prefix pair is built from strictly smaller prefix pairs) and **overlapping subproblems** (those smaller pairs recur across branches). Filling each of the `(m+1) × (n+1)` cells once turns the exponential search of section 5 into a pair of nested loops.
- **Complexity:** `(m+1) × (n+1)` cells, each filled in `O(1)` from three already-computed neighbours, gives **`O(m·n)` time** and **`O(m·n)` space** — reducible to **`O(min(m, n))` space** when only the length is wanted (section 7). Recovering the subsequence walks back through the table in `O(m + n)` steps.

LCS is the standard illustration that a problem with an exponential search space can collapse to a polynomial one the moment its repeated subproblems are given somewhere to be remembered.